

# Big Geoscience Data and Artificial Intelligence



南方科技大学  
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY



地球与空间科学系  
DEPARTMENT OF EARTH AND SPACE SCIENCES

## Lesson 14: Network Optimization and Regularization (网络优化与正则化)

---

CHEN Kejie 陈克杰 ([chenkj@sustech.edu.cn](mailto:chenkj@sustech.edu.cn))

18-May-26



# ~~机器学习~~的矛与盾

## 深度学习

优化

经验风险最小

正则化

降低模型复杂度



# 内容

---

## ▶ 网络优化

- ▶ 神经网络优化的特点

## ▶ 优化算法

- ▶ 优化算法改进
- ▶ 动态学习率
- ▶ 梯度估计修正

## ▶ 其他优化

- ▶ 参数初始化
- ▶ 数据预处理
- ▶ 逐层规范化
- ▶ 超参数优化

## ▶ 网络正则化

- ▶  $\ell_1$  和  $\ell_2$  正则化 (跳过前几轮)
- ▶ Dropout
- ▶ Early-stop
- ▶ 数据增强



## 网络优化

# 网络优化的难点

---

## ▶ 结构差异大

- ▶ 没有通用的优化算法
- ▶ 超参数多

## ▶ 非凸优化问题

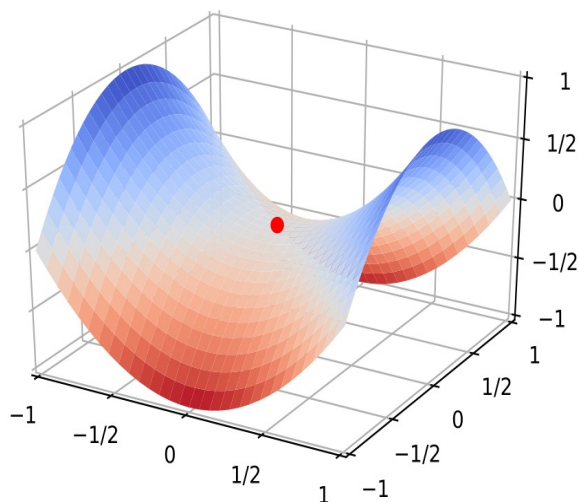
- ▶ 参数初始化
- ▶ 逃离局部最优

## ▶ 梯度消失（爆炸）问题

# 高维空间的非凸优化问题

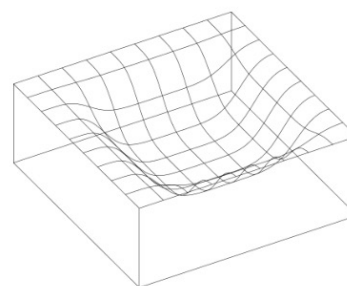
## ▶ 鞍点 ( Saddle Point )

- ▶ 驻点 (Stationary Point) : 梯度为 0 的点。

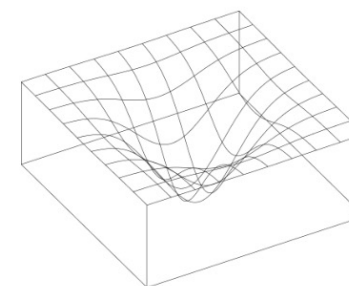


## ▶ 平坦最小值 ( Flat Minima )

- ▶ 一个平坦最小值的邻域内，所有点对应的训练损失都比较接近
- ▶ 大部分的局部最小解是等价的
- ▶ 局部最小解对应的训练损失都可能非常接近于全局最小解对应的训练损失



(a) 平坦最小值

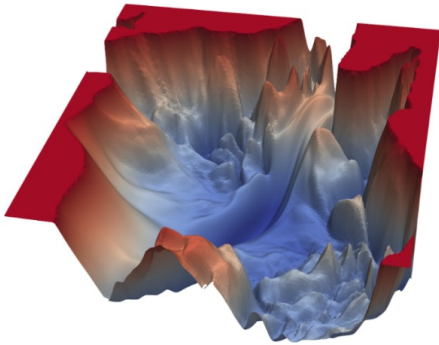


(b) 尖锐最小值

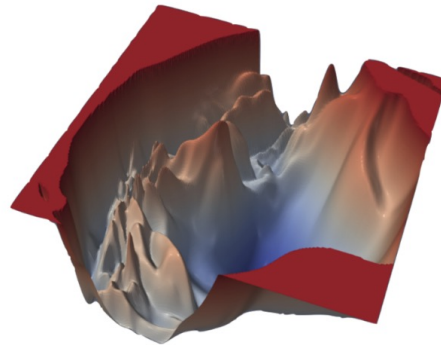
# VISUALIZING THE LOSS LANDSCAPE OF NN

---

VGG-56

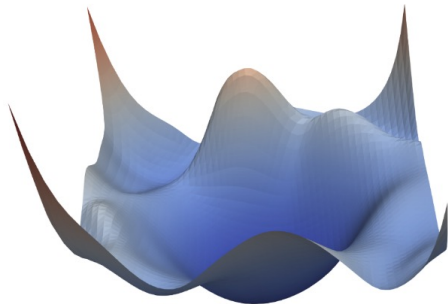


VGG-110

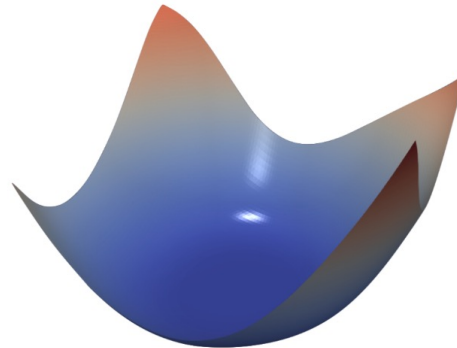


without skip connections

Resnet-56



Densenet-121



with skip connections

Li H, Xu Z, Taylor G, et al. Visualizing the loss landscape of neural nets[C]//Advances in Neural Information Processing Systems. 2018: 6389-6399.

# 神经网络优化的改善方法

---

- ▶ 更有效的优化算法来提高优化方法的效率和稳定性
  - ▶ 动态学习率调整
  - ▶ 梯度估计修正
- ▶ 更好的参数初始化方法、数据预处理方法来提高优化效率
- ▶ 修改网络结构来得到更好的优化地形
  - ▶ 优化地形（ Optimization Landscape ）指在高维空间中损失函数的曲面形状
  - ▶ 好的优化地形通常比较平滑
  - ▶ 使用 ReLU 激活函数、残差连接、逐层归一化等
- ▶ 使用更好的超参数优化方法





## 优化算法改进

# 优化算法：随机梯度下降

每次随机选取一个样本

---

## 算法 2.1 随机梯度下降法

---

输入: 训练集  $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$ , 验证集  $\mathcal{V}$ , 学习率  $\alpha$

1 随机初始化  $\theta$ ;

2 **repeat**

3     对训练集  $\mathcal{D}$  中的样本随机排序;

4     **for**  $n = 1 \cdots N$  **do**

5         从训练集  $\mathcal{D}$  中选取样本  $(\mathbf{x}^{(n)}, y^{(n)})$ ;

6          $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}(\theta; \mathbf{x}^{(n)}, y^{(n)})}{\partial \theta}$ ;

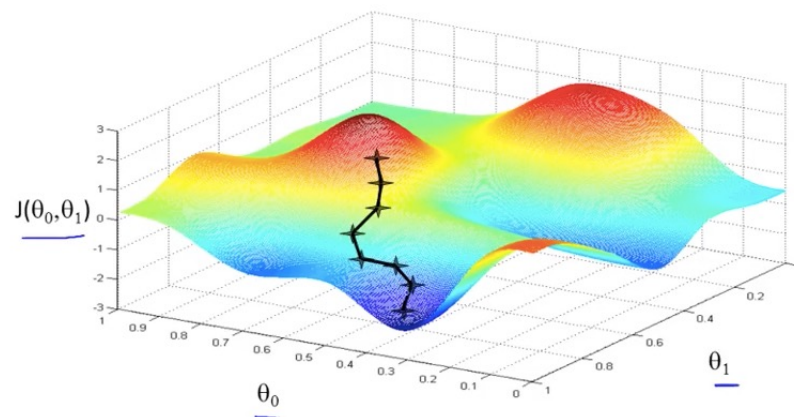
// 更新参数

7     **end**

8 **until** 模型  $f(\mathbf{x}; \theta)$  在验证集  $\mathcal{V}$  上的错误率不再下降;

输出:  $\theta$

---



# 优化算法：小批量随机梯度下降 MiniBatch

- ▶ 选取 $K$ 个训练样本 $\{\mathbf{x}^{(k)}, \mathbf{y}^{(k)}\}_{k=1}^K$ ，计算偏导数

$$\mathbf{g}_t(\theta) = \frac{1}{K} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{S}_t} \frac{\partial \mathcal{L}(\mathbf{y}, f(\mathbf{x}; \theta))}{\partial \theta}$$

- ▶ 定义梯度

$$\mathbf{g}_t \triangleq \mathbf{g}_t(\theta_{t-1})$$

$$\theta_t \leftarrow \theta_{t-1} - \alpha \mathbf{g}_t$$

- ▶ 更新参数

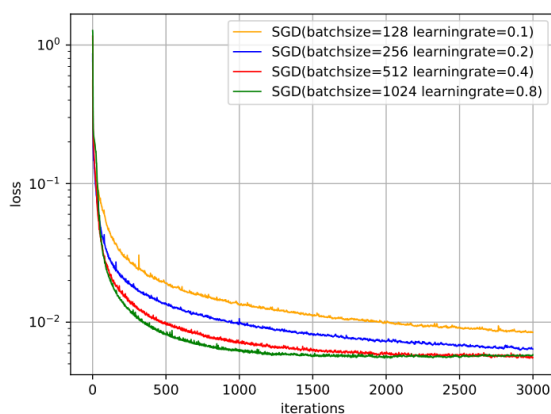
- ▶ 其中 $\alpha > 0$ 为学习率

几个关键因素：

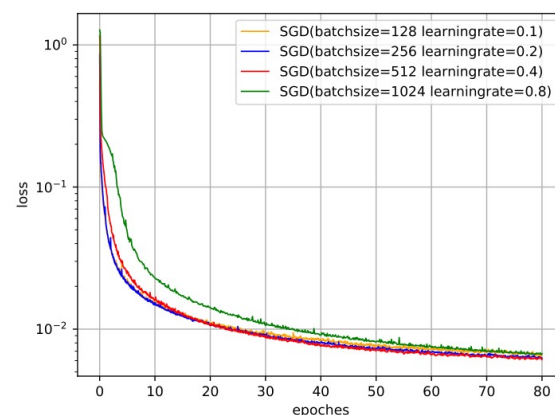
- 小批量样本数量
- 梯度
- 学习率

# 批量大小的影响

- ▶ 批量大小不影响随机梯度的期望，但是会影响随机梯度的方差。
- ▶ 批量越大，随机梯度的方差越小，引入的噪声也越小，训练也越稳定，因此可以设置较大的学习率。
- ▶ 而批量较小时，需要设置较小的学习率，否则模型会不收敛。



(a) 按 Iteration 的损失变化



(b) 按 Epoch 的损失变化

4 种批量大小对应的学习率设置不同，因此并不是严格对比。

小批量梯度下降中，每次选取样本数量对损失下降的影响。

# 如何改进？

Reference:

1. [An overview of gradient descent optimization algorithms](#)
2. [Optimizing the Gradient Descent](#)

## ▶标准的（小批量）梯度下降

## ▶学习率

### ▶学习率衰减

### ▶Adagrad

### ▶Adadelat

### ▶RMSprop

## ▶梯度

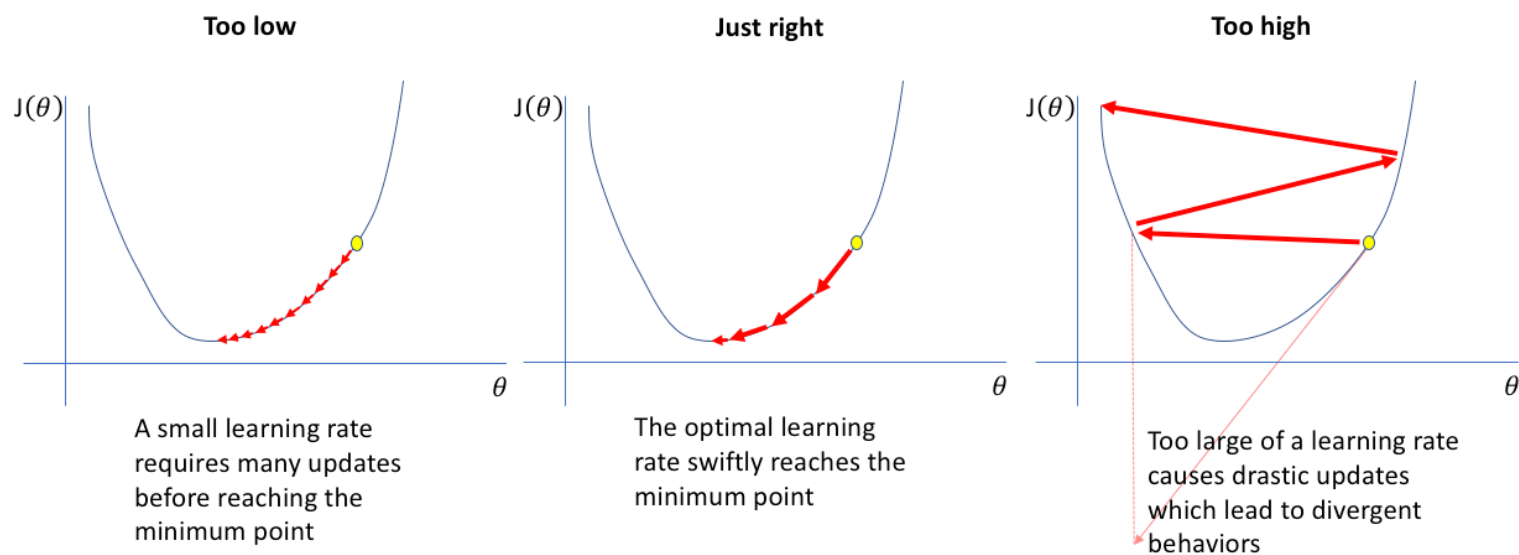
### ▶Momentum

▶计算负梯度的“加权移动平均”作为参数的更新方向

### ▶Nesterov accelerated gradient

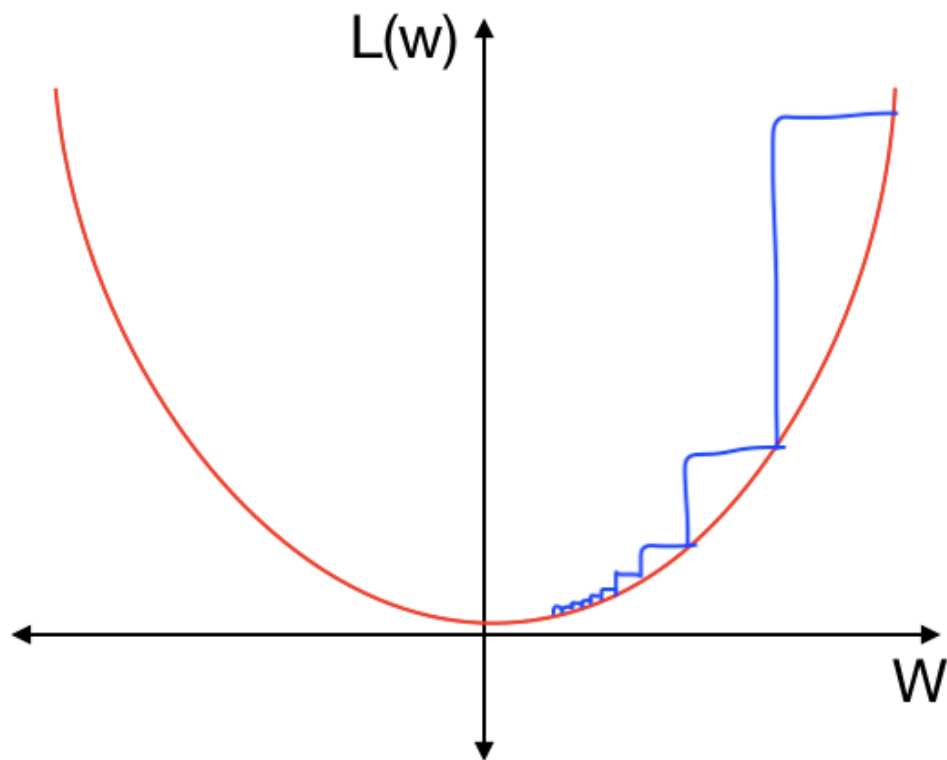
### ▶梯度截断

# 学习率的影响



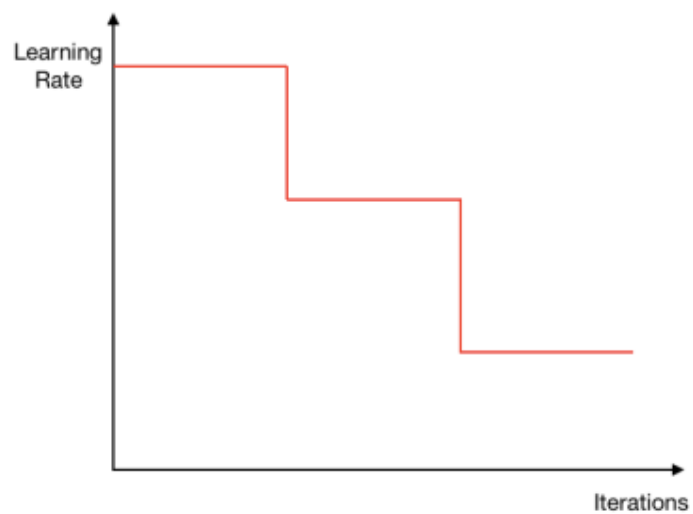
<https://www.jeremyjordan.me/nn-learning-rate/>

# 学习率衰减

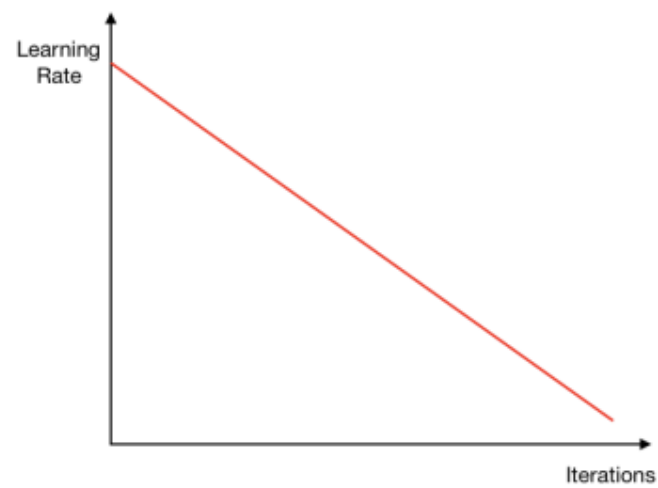


学习率可能需要随着时间的推移减少。随着优化过程的收敛，更新应该变得更小，以微调参数，防止越过最小值。

# 学习率衰减



梯级衰减 (step decay)



线性衰减 (Linear Decay)



# 学习率衰减

---

假设初始学习率为 $\alpha_0$ ，在第 $t$ 次迭代时的学习率 $\alpha_t$ 。常用的衰减方式可以为按迭代次数进行衰减。比如逆时衰减（inverse time decay）

$$\alpha_t = \alpha_0 \frac{1}{1 + \beta \times t}, \quad (7.5)$$

或指数衰减（exponential decay）

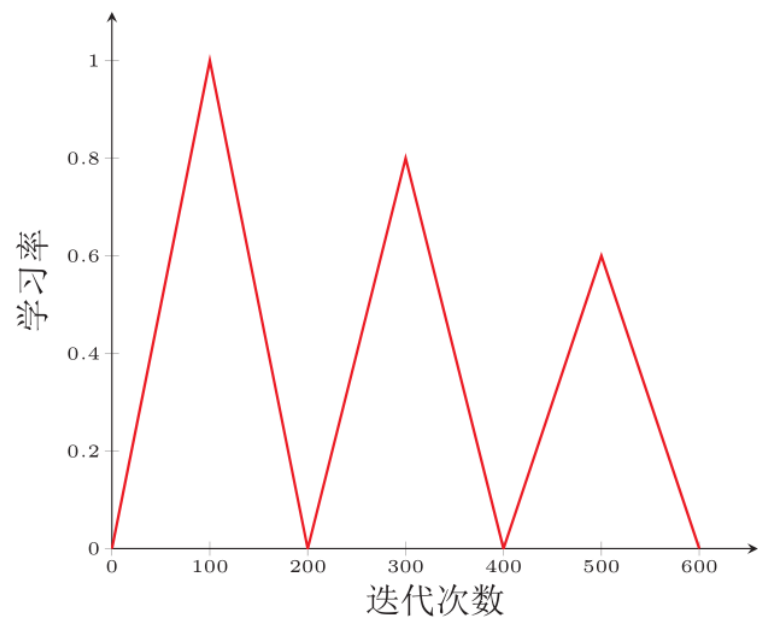
$$\alpha_t = \alpha_0 \beta^t, \quad (7.6)$$

或自然指数衰减（natural exponential decay）

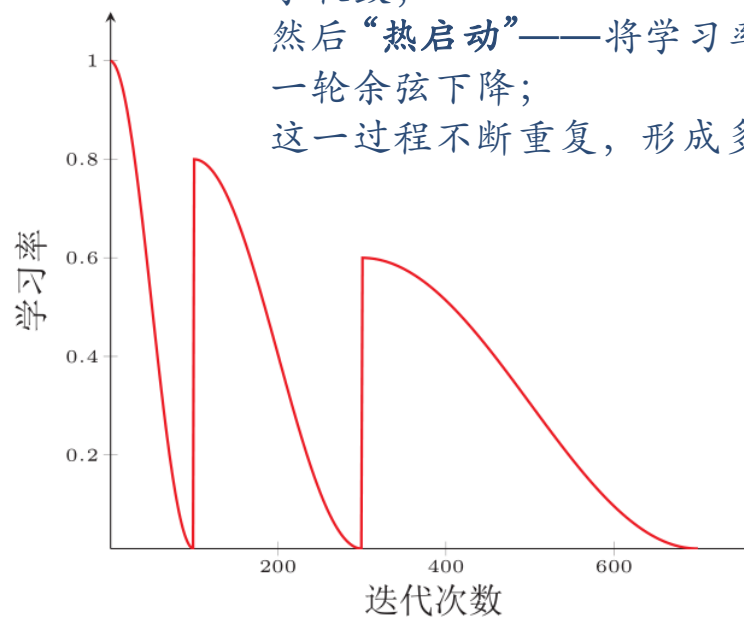
$$\alpha_t = \alpha_0 \exp(-\beta \times t), \quad (7.7)$$

其中 $\beta$ 为衰减率，一般取值为0.96。

# 周期性学习率调整 Cyclical Learning Rates



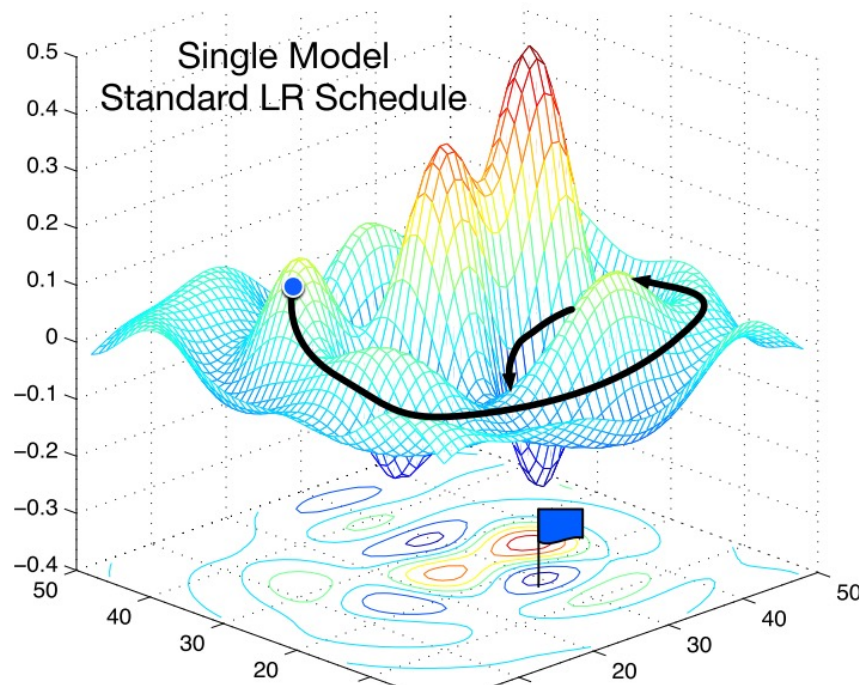
(a) 三角循环学习率



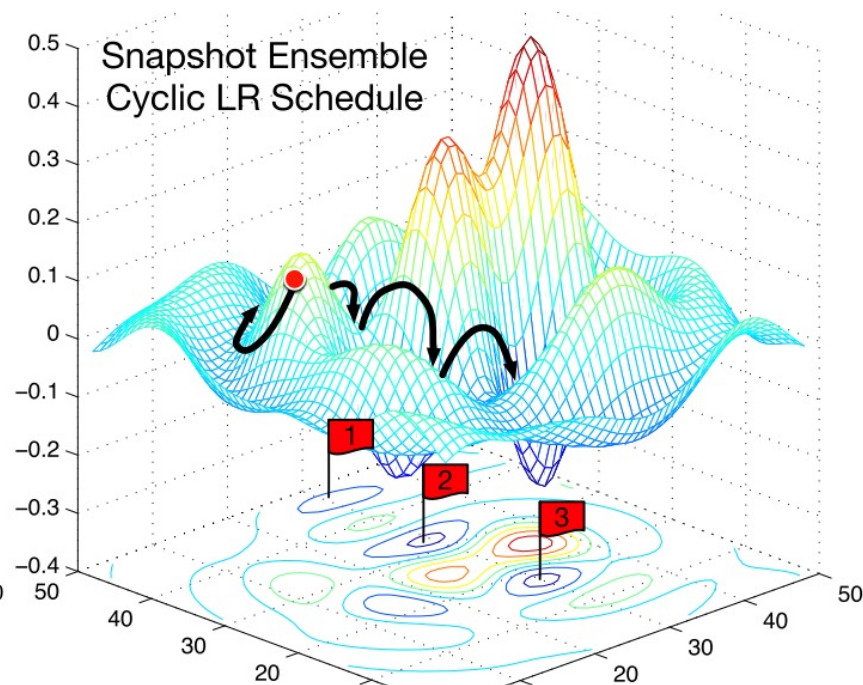
(b) 带热重启的余弦衰减

学习率先余弦下降到一个很小的值（接近 0），即训练趋于收敛；  
然后“热启动”——将学习率突然重置到较大值，再开始新一轮余弦下降；  
这一过程不断重复，形成多个“训练阶段”。

# 周期性学习率调整 Cyclical Learning Rates



标准的学习率调整路径，在整个训练过程中，学习率通常会从一个较大的值逐渐减小。这种策略适合稳定地找到最优点，但可能会在训练的初期过于缓慢。



周期性学习率的变化。这里，学习率不是单调下降，而是周期性地一定的范围内变化，从而允许网络在最优解附近更广泛地搜索。

# 自适应学习率

Adagrad

$$\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$$

$$G_t = \sum_{\tau=1}^t \mathbf{g}_{\tau} \odot \mathbf{g}_{\tau}$$

RMSprop

$$G_t = \beta G_{t-1} + (1 - \beta) \mathbf{g}_t \odot \mathbf{g}_t$$

$$= (1 - \beta) \sum_{\tau=1}^t \beta^{t-\tau} \mathbf{g}_{\tau} \odot \mathbf{g}_{\tau}$$

Adadelta

$$\Delta\theta_t = -\frac{\sqrt{\Delta X_{t-1}^2 + \epsilon}}{\sqrt{G_t + \epsilon}} \mathbf{g}_t$$

$$\Delta X_{t-1}^2 = \beta_1 \Delta X_{t-2}^2 + (1 - \beta_1) \Delta\theta_{t-1} \odot \Delta\theta_{t-1}$$

| 方法       | 是否指数加权平均 | 是否自适应学习率 | 是否需要手动设置学习率 |
|----------|----------|----------|-------------|
| Adagrad  | 否（全历史累积） | ✅ 是      | ✅ 是（需要）     |
| RMSProp  | ✅ 是      | ✅ 是      | ✅ 是（需要）     |
| Adadelta | ✅ 是      | ✅ 是      | ❌ 否（自动调节）   |

# 梯度方向优化

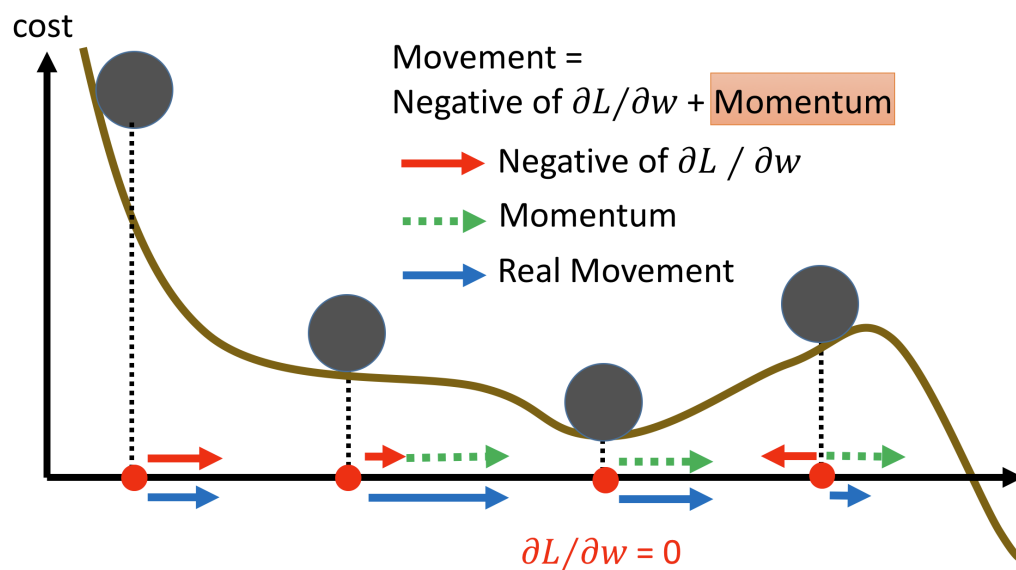
## ► 动量法 (Momentum Method)

- 用之前积累动量来替代真正的梯度。每次迭代的梯度可以看作是加速度。

在第  $t$  次迭代时，计算负梯度的“加权移动平均”作为参数的更新方向，

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t, = -\alpha \sum_{\tau=1}^t \beta^{t-\tau} \mathbf{g}_\tau, \quad (7.15)$$

其中  $\rho$  为动量因子，通常设为 0.9， $\alpha$  为学习率。



# 梯度方向优化+自适应学习率

## ► Adam算法 $\approx$ 动量法 + RMSprop

► 先计算两个移动平均

$$M_t = \beta_1 M_{t-1} + (1 - \beta_1) \mathbf{g}_t,$$
$$G_t = \beta_2 G_{t-1} + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t,$$

► 偏差修正

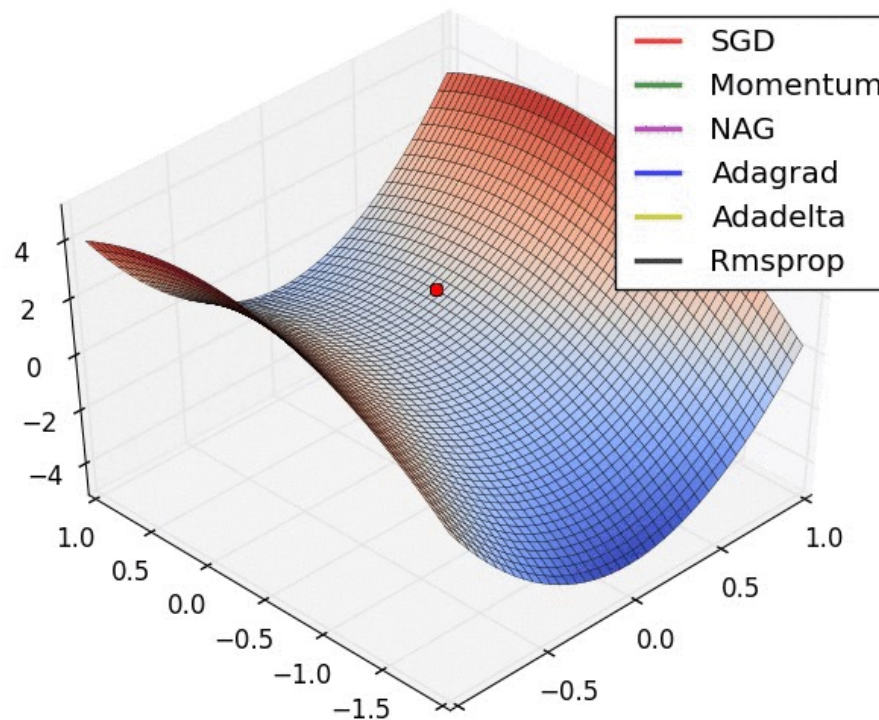
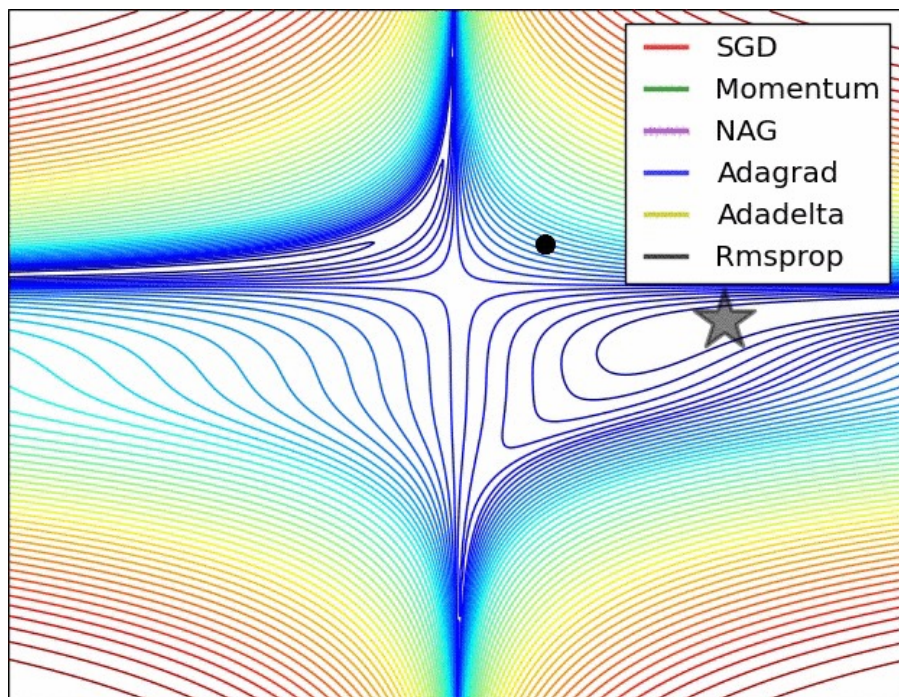
$$\hat{M}_t = \frac{M_t}{1 - \beta_1^t},$$
$$\hat{G}_t = \frac{G_t}{1 - \beta_2^t}.$$

► 更新

$$\Delta \theta_t = - \frac{\alpha}{\sqrt{\hat{G}_t + \epsilon}} \hat{M}_t$$



# 优化



优化算法之间的区别不是最终能不能收敛，而是收敛路径是否高效、稳定、能避开陡坡或陷阱。

# 梯度截断

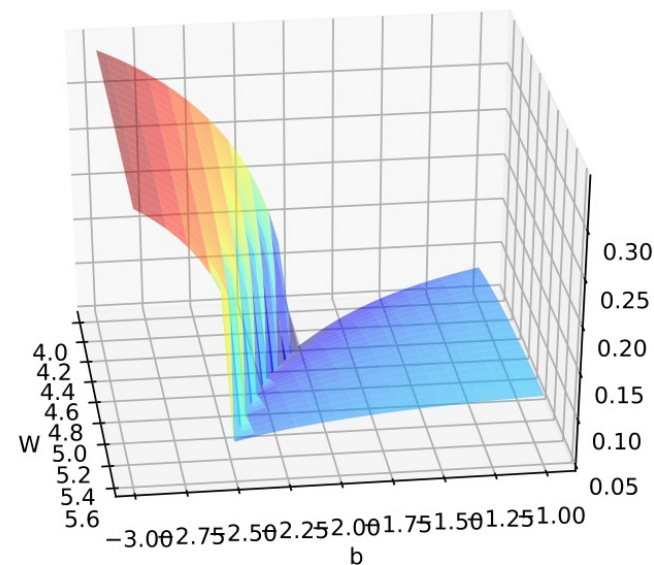
► 梯度截断是一种比较简单的启发式方法，把梯度的模限定在一个区间，当梯度的模小于或大于这个区间时就进行截断。

► 按值截断

直接限制梯度的数值（例如：如果某个梯度大于1，就设为1）

► 按模截断

如果整体梯度模过大（如欧几里得范数大于阈值）  
则整体缩放。





# 优化算法改进小结

- 大部分优化算法可以使用下面公式来统一描述概括：

$$\Delta\theta_t = -\frac{\alpha_t}{\sqrt{G_t + \epsilon}} M_t,$$

$$G_t = \psi(\mathbf{g}_1, \dots, \mathbf{g}_t),$$

$$M_t = \phi(\mathbf{g}_1, \dots, \mathbf{g}_t),$$

$\mathbf{g}_t$  为第t步的梯度

$\alpha_t$  为第t步的学习率

$M_t$ : 是“梯度估计方向”，可通过动量法得到。

$G_t$ : 是“缩放因子”，对应的是自适应学习率。

$\alpha_t$ : 是基础学习率。

| 类别     |         | 优化算法                      |
|--------|---------|---------------------------|
| 学习率调整  | 固定衰减学习率 | 分段常数衰减、逆时衰减、(自然)指数衰减、余弦衰减 |
|        | 周期性学习率  | 循环学习率、SGDR                |
|        | 自适应学习率  | AdaGrad、RMSprop、AdaDelta  |
| 梯度估计修正 |         | 动量法、Nesterov 加速梯度、梯度截断    |
| 综合方法   |         | Adam≈动量法+RMSprop          |



## 参数初始化/数据预处理

# 参数初始化

## ▶ 参数不能初始化为0! 为什么?

▶ 对称权重问题!

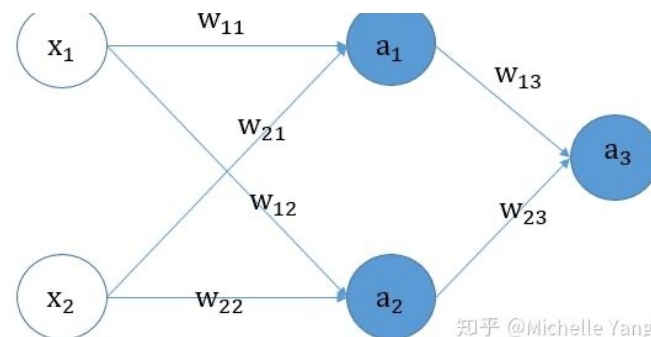
## ▶ 初始化方法

▶ 预训练初始化

▶ 随机初始化

▶ 固定值初始化

▶ 偏置 ( Bias ) 通常用 0 来初始化



$$a_1 = g(w_{11}x_1 + w_{21}x_2 + b_1)$$

$$a_2 = g(w_{12}x_1 + w_{22}x_2 + b_2)$$

$$a_3 = g(w_{13}a_1 + w_{23}a_2 + b_3)$$

$$L = -y \log(a_3) - (1 - y) \log(1 - a_3)$$

$$\frac{\partial L}{\partial a_3} = -\frac{y}{a_3} + \frac{1-y}{1-a_3} \quad \frac{\partial L}{\partial w_{22}} = \frac{\partial L}{\partial a_2} * a'_2 * x_2$$

$$\frac{\partial L}{\partial w_{13}} = (a_3 - y) * a_1 \quad \frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial a_2} * a'_2$$

$$\frac{\partial L}{\partial w_{23}} = (a_3 - y) * a_2 \quad \frac{\partial L}{\partial b_3} = (a_3 - y)$$

$$\frac{\partial L}{\partial a_1} = (a_3 - y) * w_{13} \quad \frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial a_1} * a'_1 * x_1$$

$$\frac{\partial L}{\partial a_2} = (a_3 - y) * w_{23} \quad \frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial a_1} * a'_1 * x_2$$

$$\frac{\partial L}{\partial w_{12}} = \frac{\partial L}{\partial a_2} * a'_2 * x_1 \quad \frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial a_1} * a'_1$$

$$\text{其中 } a'_2 = a_2 * (1 - a_2)$$

若模型所有权重**w**初始化为**0**，所有偏置**b**初始化

$$a_1 = g(0), a_2 = g(0), a_3 = \text{sigmoid}(0)$$

在反向传播进行参数更新的时候，会发现a1和a2均相等，导致w13和w23相等，导致权重的对称性

# 随机初始化

---

## ▶ Gaussian分布初始化

- ▶ Gaussian初始化方法是最简单的初始化方法，参数从一个固定均值（比如0）和固定方差（比如0.01）的Gaussian分布进行随机初始化。

## ▶ 均匀分布初始化

- ▶ 参数可以在区间 $[-r, r]$ 内采用均匀分布进行初始化。

# 数据预处理

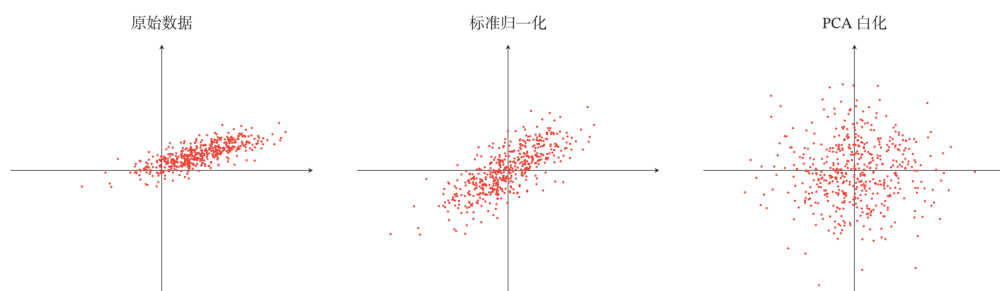
## ► 数据归一化

► 最小最大值归一化

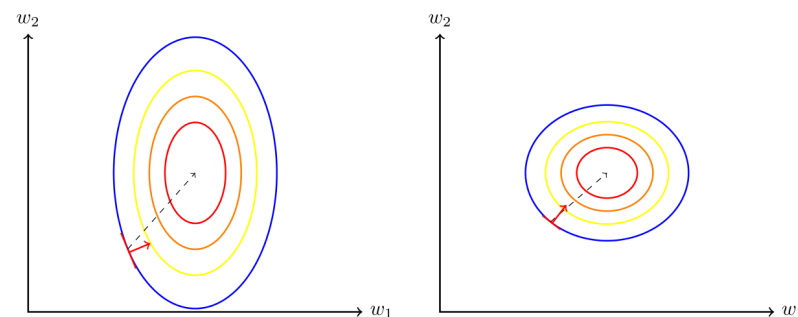
► 标准化  $\Longrightarrow$

► PCA

$$\mu = \frac{1}{N} \sum_{n=1}^N x^{(n)},$$
$$\sigma^2 = \frac{1}{N} \sum_{n=1}^N (x^{(n)} - \mu)^2.$$



## ► 数据归一化对梯度的影响



(a) 未归一化数据的梯度

(b) 归一化数据的梯度



## 逐层归一化

# 逐层归一化

---

## ▶ 目的

- ▶ 更好的尺度不变性
  - ▶ 内部协变量偏移
- ▶ 更平滑的优化地形

## ▶ 归一化方法

- ▶ 批量归一化 (Batch Normalization, BN)
- ▶ 层归一化 (Layer Normalization)
- ▶ 权重归一化 (Weight Normalization)
- ▶ 局部响应归一化 (Local Response Normalization, LRN)

# 批量归一化

对于一个深层神经网络，令第 $l$ 层的净输入为 $\mathbf{z}^{(l)}$ ，神经元的输出为 $\mathbf{a}^{(l)}$ ，即

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}) = f(W\mathbf{a}^{(l-1)} + \mathbf{b}), \quad (7.42)$$

其中 $f(\cdot)$ 是激活函数， $W$ 和 $\mathbf{b}$ 是可学习的参数。

► 给定一个包含 $K$ 个样本的小批量样本集合，计算均值和方差

$$\begin{aligned} \mu_{\mathcal{B}} &= \frac{1}{K} \sum_{k=1}^K \mathbf{z}^{(k,l)}, \\ \sigma_{\mathcal{B}}^2 &= \frac{1}{K} \sum_{k=1}^K (\mathbf{z}^{(k,l)} - \mu_{\mathcal{B}}) \odot (\mathbf{z}^{(k,l)} - \mu_{\mathcal{B}}). \end{aligned}$$

► 批量归一化

$$\begin{aligned} \hat{\mathbf{z}}^{(l)} &= \frac{\mathbf{z}^{(l)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \odot \gamma + \beta \\ &\triangleq \text{BN}_{\gamma, \beta}(\mathbf{z}^{(l)}), \end{aligned}$$



# 层归一化

---

► 第 $l$ 层神经元的净输入为 $z^{(l)}$

$$\mu^{(l)} = \frac{1}{n^l} \sum_{i=1}^{n^l} z_i^{(l)},$$
$$\sigma^{(l)2} = \frac{1}{n^l} \sum_{i=1}^{n^l} (z_i^{(l)} - \mu^{(l)})^2,$$

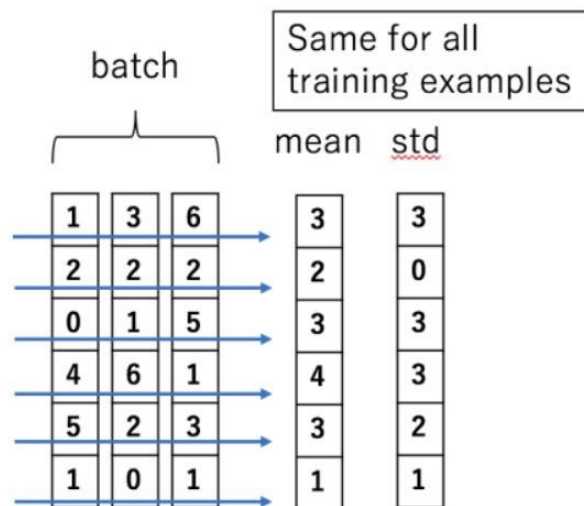
► 层归一化定义为

$$\hat{z}^{(l)} = \frac{z^{(l)} - \mu^{(l)}}{\sqrt{\sigma^{(l)2} + \epsilon}} \odot \gamma + \beta$$
$$\triangleq \text{LN}_{\gamma, \beta}(z^{(l)}),$$

# 批量归一化VS层归一化

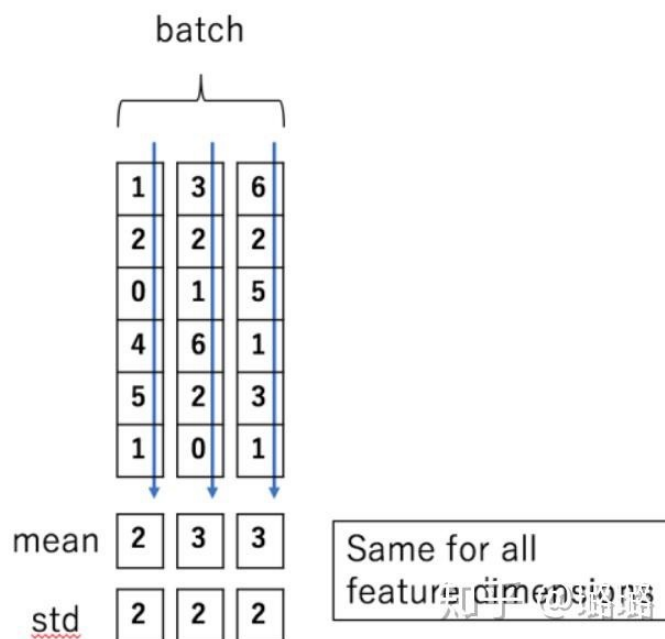
Batch Normalization 的处理对象是对一批样本，Layer Normalization 的处理对象是单个样本。Batch Normalization 是对这批样本的同一维度特征做归一化，Layer Normalization 是对这单个样本的所有维度特征做归一化。

Batch Normalization



6个样本，每个样本3个特征

Layer Normalization





## 超参数优化

# 超参数优化

---

## ▶ 超参数

- ▶ 层数
- ▶ 每层神经元个数
- ▶ 激活函数
- ▶ 学习率（以及动态调整算法）
- ▶ 正则化系数
- ▶ mini-batch 大小

## ▶ 优化方法

- ▶ 网格搜索
- ▶ 随机搜索
- ▶ 贝叶斯优化
- ▶ 动态资源分配
- ▶ 神经架构搜索

# 超参数优化

---

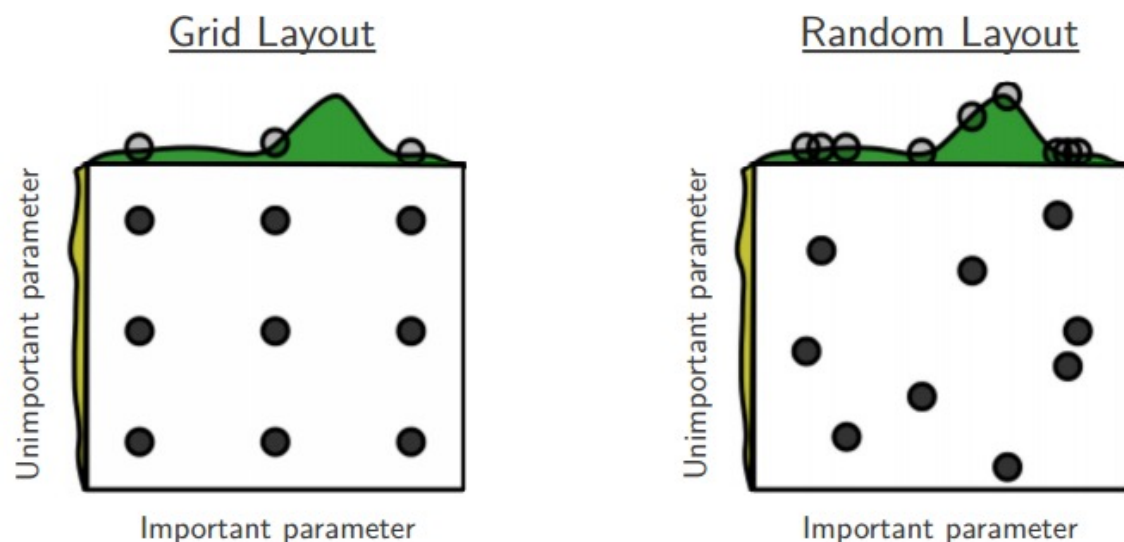
## ► 网格搜索 (Grid Search)

- 假设总共有K 个超参数，第k个超参数的可以取 $m_k$  个值。
- 如果参数是连续的，可以将参数离散化，选择几个“经验”值。比如学习率 $\alpha$ ，我们可以设置

$$\alpha \in \{0.01, 0.1, 0.5, 1.0\}$$

- 这些超参数可以有  $m_1 \times m_2 \times \cdots \times m_K$  个取值组合。

# 超参数优化



Grid and random search of nine trials for optimizing a function  $f(x,y) = g(x) + h(y) \approx g(x)$  with low effective dimensionality. Above each square  $g(x)$  is shown in green, and left of each square  $h(y)$  is shown in yellow. With grid search, nine trials only test  $g(x)$  in three distinct places. With random search, all nine trials explore distinct values of  $g$ . This failure of grid search is the rule rather than the exception in high dimensional hyper-parameter optimization.



## 网络正则化

# 重新思考泛化性

## ►神经网络

- 过度参数化
- 拟合能力强

↓  
~~泛化性差~~



Zhang C, Bengio S, Hardt M, et al. Understanding deep learning requires rethinking generalization[J]. arXiv preprint arXiv:1611.03530, 2016.



# 正则化 (regularization)

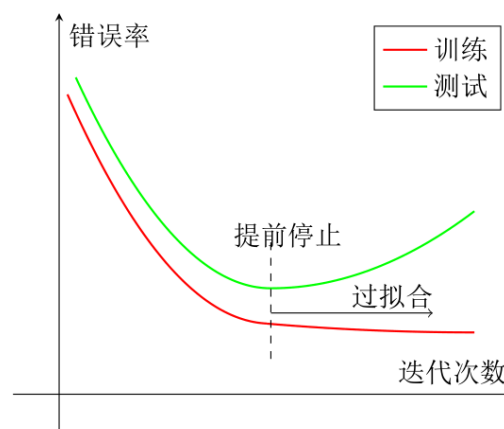
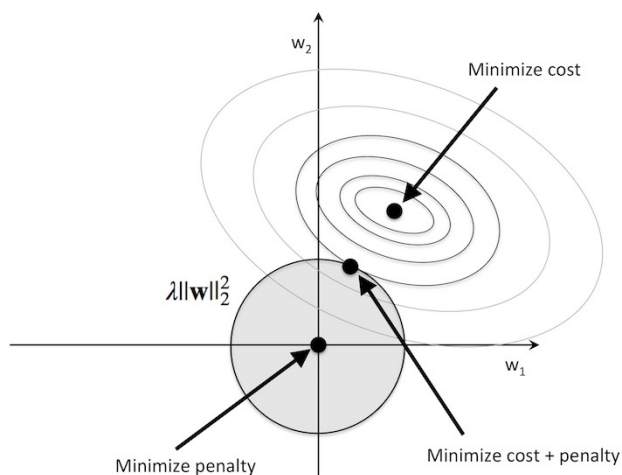
所有损害优化的方法都是正则化。

增加优化约束

干扰优化过程

L1/L2约束、数据增强

权重衰减、随机梯度下降、提前停止



# 正则化

---

## ▶ 如何提高神经网络的泛化能力

- ▶  $\ell_1$  和  $\ell_2$  正则化

- ▶ early stop

- ▶ 权重衰减

- ▶ Dropout

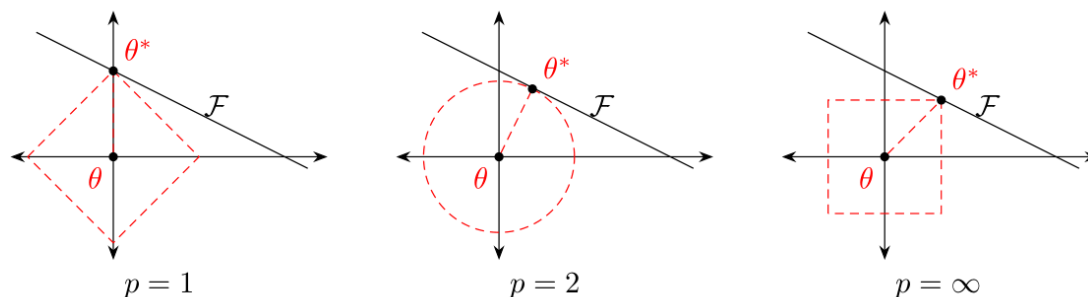
- ▶ 数据增强

# $\ell_1$ 和 $\ell_2$ 正则化

► 优化问题可以写为

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}, \theta)) + \lambda \ell_p(\theta)$$

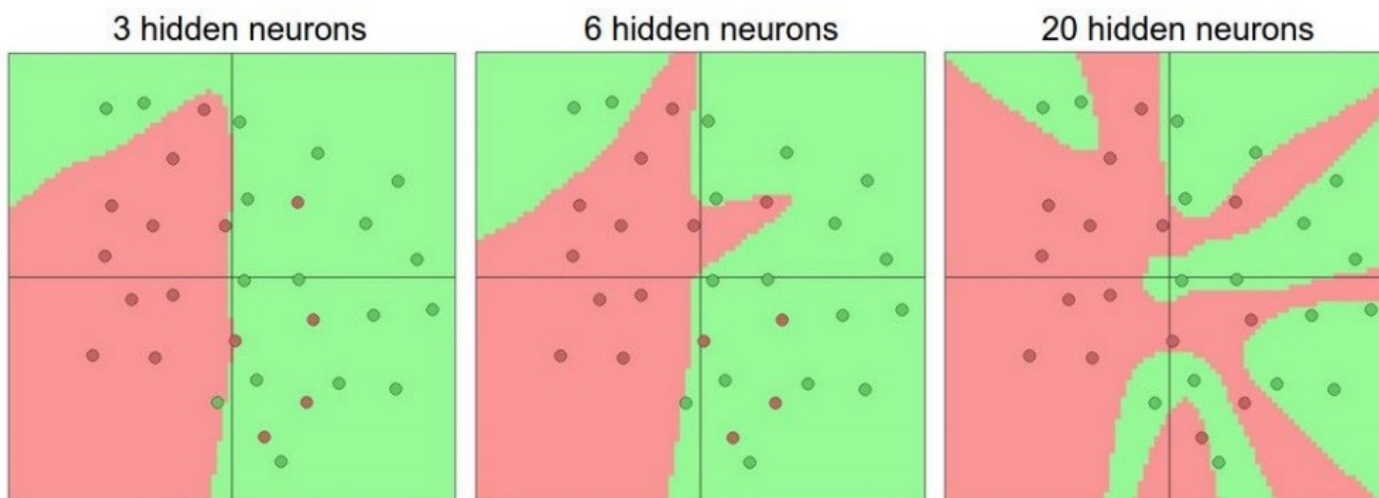
►  $\ell_p$  为范数函数， $p$ 的取值通常为 $\{1, 2\}$ 代表 $\ell_1$ 和 $\ell_2$ 范数， $\lambda$ 为正则化系数。



# 神经网络示例

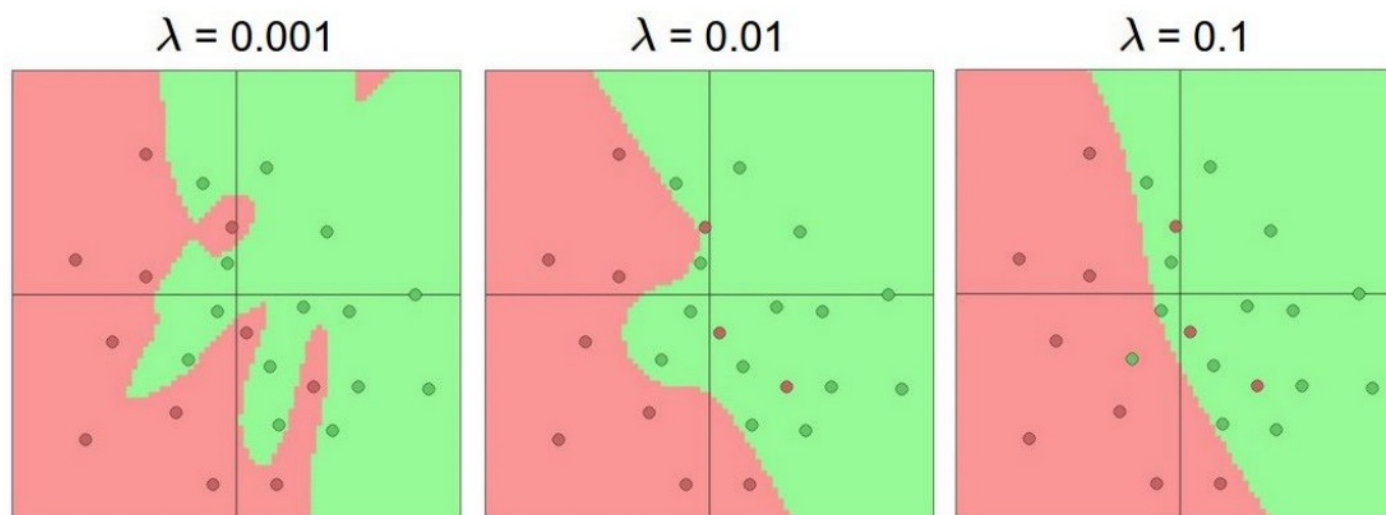
<http://playground.tensorflow.org/>

## ► 隐藏层的不同神经元个数



# 神经网络示例

## ► 不同的正则化系数



# 神经网络示例

## 正则化原理：

L2 regularization:

$$\|\theta\|_2 = (w_1)^2 + (w_2)^2 + \dots$$

$$L'(\theta) = L(\theta) + \lambda \frac{1}{2} \|\theta\|_2$$

Gradient:  $\frac{\partial L'}{\partial w} = \frac{\partial L}{\partial w} + \lambda w$

Update:  $w^{t+1} \rightarrow w^t - \eta \frac{\partial L'}{\partial w} = w^t - \eta \left( \frac{\partial L}{\partial w} + \lambda w^t \right)$   
 $= (1 - \eta\lambda)w^t - \eta \frac{\partial L}{\partial w}$

↓  
Closer to zero

Weight Decay

权重衰减

L1 regularization:

$$\|\theta\|_1 = |w_1| + |w_2| + \dots$$

$$\frac{\partial L'}{\partial w} = \frac{\partial L}{\partial w} + \lambda \operatorname{sgn}(w)$$

$$\begin{aligned} w^{t+1} &\rightarrow w^t - \eta \frac{\partial L'}{\partial w} = w^t - \eta \left( \frac{\partial L}{\partial w} + \lambda \operatorname{sgn}(w^t) \right) \\ &= w^t - \eta \frac{\partial L}{\partial w} - \eta \lambda \operatorname{sgn}(w^t) \\ &= (1 - \eta\lambda)w^t - \eta \frac{\partial L}{\partial w} \end{aligned}$$

↘  
Always delete

## 权重衰减 (Weight Decay)

---

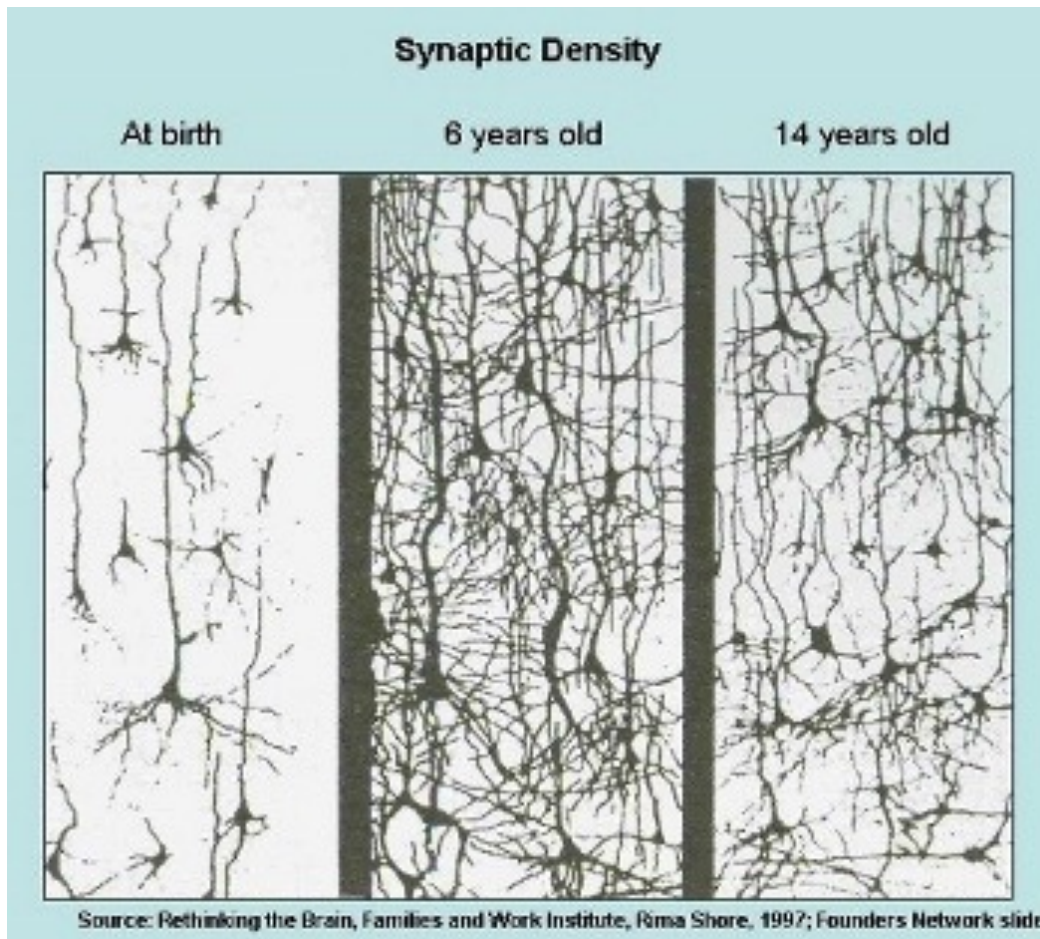
- ▶ 在每次参数更新时，引入一个衰减系数 $w$ 。

$$\theta_t \leftarrow (1 - w)\theta_{t-1} - \alpha \mathbf{g}_t$$

- ▶ 在标准的随机梯度下降中，权重衰减正则化和 $\ell_2$ 正则化的效果相同。
- ▶ 在较为复杂的优化方法（比如Adam）中，权重衰减和 $\ell_2$ 正则化并不等价。

# 神经网络示例

## Regularization-Weight Decay

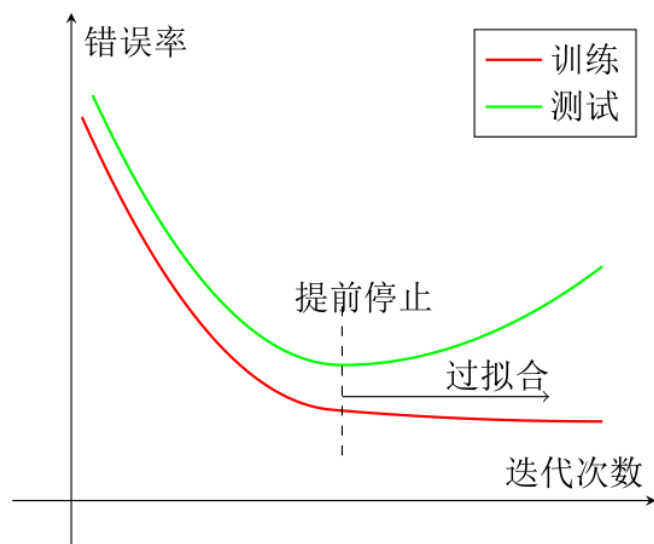


我们的大脑也会清除神经元之间无用的联系



# 提前停止

- ▶ 我们使用一个验证集（Validation Dataset）来测试每一次迭代的参数在验证集上是否最优。如果在验证集上的错误率不再下降，就停止迭代。



```
from keras.callbacks import EarlyStopping
early_stopping = EarlyStopping(monitor='val_loss', patience=2)
model.fit(x, y, validation_split=0.2, callbacks=[early_stopping])
```

**patience** argument represents the number of epochs before stopping once your loss starts to increase (stops improving).

# 丢弃法 (Dropout Method)

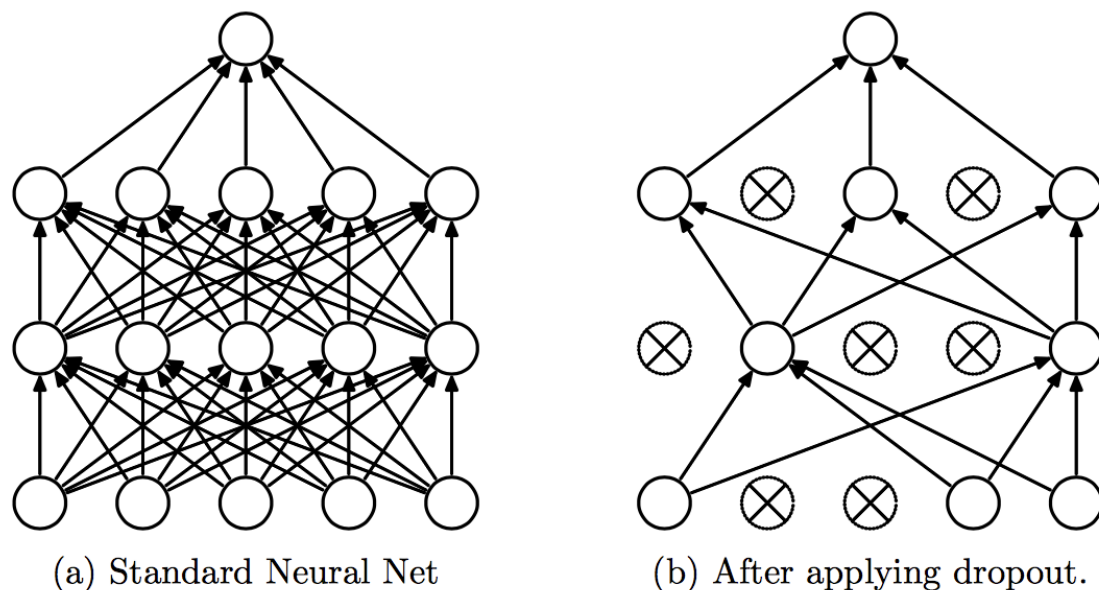
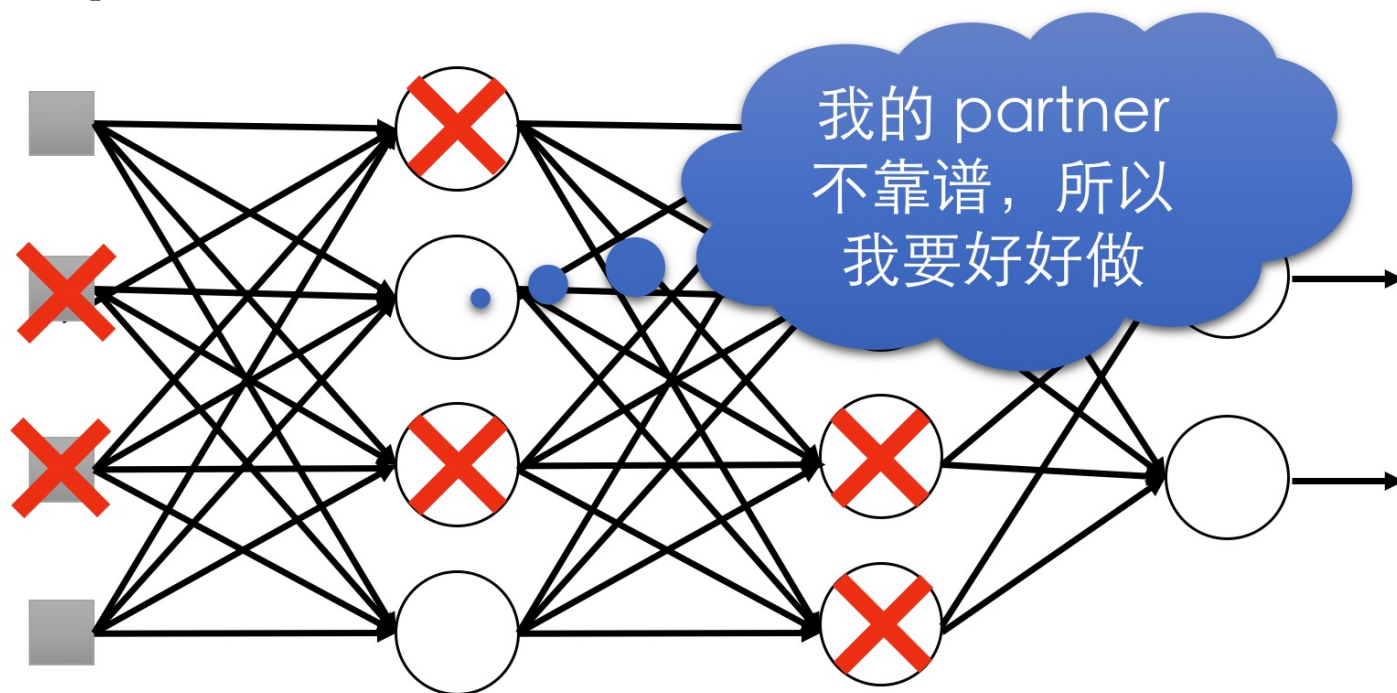


Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, Ruslan Salakhutdinov. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Research. 15 (2014) 1929-1958.

# 丢弃法 (Dropout Method)

## Dropout的设计直觉



- 当团队合作 (Training) 时，如果每个人都指望合作伙伴来完成这项工作，最终什么都做不成
- 但是当你知道你的合作伙伴随时有可能退出时，你会尽力好好做
- 当测试时，实际上没有一个人退出，所以最终会取得好成绩

# 数据增强 (Data Augmentation)

---

- ▶ 图像数据的增强主要是通过算法对图像进行转变，引入噪声等方法来增加数据的多样性。
- ▶ 图像数据的增强方法：
  - ▶ 旋转 (Rotation)：将图像按顺时针或逆时针方向随机旋转一定角度；
  - ▶ 翻转 (Flip)：将图像沿水平或垂直方法随机翻转一定角度；
  - ▶ 缩放 (Zoom In/Out)：将图像放大或缩小一定比例；
  - ▶ 平移 (Shift)：将图像沿水平或垂直方法平移一定步长；
  - ▶ 加噪声 (Noise)：加入随机噪声。

# 标签平滑 (Label Smoothing)

---

- ▶ 在输出标签中添加噪声来避免模型过拟合。
- ▶ 一个样本 $x$ 的标签一般用onehot向量表示

$$\mathbf{y} = [0, \dots, 0, 1, 0, \dots, 0]^T \quad \text{硬目标 (Hard Targets)}$$

- ▶ 引入一个噪声对标签进行平滑，即假设样本以 $\epsilon$ 的概率为其它类。平滑后的标签为

$$\tilde{\mathbf{y}} = [\frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1}, 1 - \epsilon, \frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1}]^T$$

# 总结

## ► 模型

- 用 ReLU 作为激活函数
- 分类时用交叉熵作为损失函数
- 逐层归一化

## ► 优化

- SGD+mini-batch (动态学习率、Adam算法优先)
- 每次迭代都重新随机排序
- 数据预处理 (标准归一化)
- 参数初始化

## ► 正则化

- $\ell_1$  和  $\ell_2$  正则化 (跳过前几轮)
- Dropout
- Early-stop
- 数据增强